

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Generici II

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

tipi generici parte II

Trova le differenze

```
static public void esamina(ArrayList<Frutto> frutti)
{
    // ...
}
```

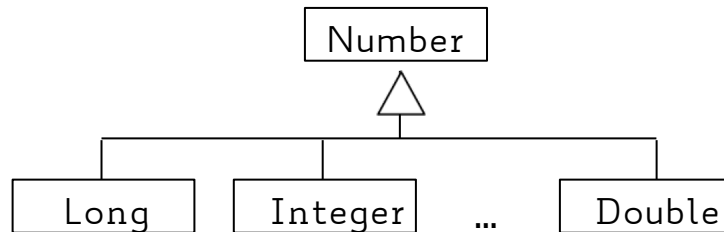
Accetta un ArrayList<Arancia>? Perché?

Non accetta un ArrayList<Arancia> perché ArrayList<Arancia> non è un sottotipo di ArrayList<Frutta>

```
static public <T extends Frutto> void esamina(ArrayList<T> frutti)
{
    // ...
}
```

Indica che **T** può essere di tipo Frutto **OPPURE** un suo sottotipo

Per le classi generiche non vale l'ereditarietà dei tipi generici



- Ad esempio, `ArrayList<Integer>` non è di tipo `ArrayList<Number>` o `ArrayList<Object>`:

```
ArrayList<Number> listaDiNumeri
    = new ArrayList<Integer>();
```

Type mismatch: cannot convert from `ArrayList<Integer>` to `ArrayList<Number>`

1 quick fix available:

[Change type of 'listaDiNumeri' to 'ArrayList<Integer>'](#)

Press 'F2' for focus

- Ma rimane comunque l'ereditarietà tra classi:

```
List<Integer> listaDiNumeri
    = new ArrayList<Integer>();
```

Per rendere "sicura" la collection a tempo di compilazione (1)

- Se fosse permesso fare l'**upcasting** tra tipi generici si avrebbero situazioni imprevedibili:

```
public class ViolenzaSuUnArrayList
{
    public static void main(String[] args)
    {
        ArrayList<Mela> mele = new ArrayList<Mela>();

        prendiFrutta(mele);

        // qui avrei una lista di mele con una pera!!!
        System.out.println(mele.toString());
    }

    public static void prendiFrutta(ArrayList<Frutto> frutti)
    {
        frutti.add(new Pera());
    }
}
```

Per fortuna!
ArrayList<Mela> non è un
sottotipo di ArrayList<Frutto>

- Il **controllo di consistenza di tipo** viene effettuato solo a tempo di compilazione

Per rendere "sicura" la collection a tempo di compilazione (2)

- E' possibile permettere il passaggio di sottotipi di Frutto utilizzando la sintassi **T extends Frutto**:

```
public class ViolenzaSuUnArrayList
{
    public static void main(String[] args)
    {
        ArrayList<Mela> mele = new ArrayList<Mela>();

        prendiFrutta(mele);

        // qui avrei una lista di mele con una pera!!!
        System.out.println(mele.toString());
    }

    public static <T extends Frutto> void prendiFrutta(ArrayList<T> frutti)
    {
        frutti.add(new Pera());
    }
}
```

Errore di compilazione: 1) è vero che Pera extends Frutto, ma 2) il controllo di consistenza viene fatto sul simbolo T e non su qualsiasi tipo occorre nel corpo del metodo generico.

- Tuttavia non è possibile "aggiungere" una pera a un elenco di mele...

Differenze tra array e collection

- L'**upcasting** è invece possibile ***a tempo di compilazione*** con gli array:

```
public class ViolenzaSuUnArray
{
    public static void main(String[] args)
    {
        Mela[] mele = new Mela[] { new Mela(), new Mela() };

        prendiFrutta(mele);

        // qui ho un array di mele con una pera!!!
        System.out.println(Arrays.toString(mele));
    }

    public static void prendiFrutta(Frutto[] frutti)
    {
        frutti[1] = new Pera();
    }
}
```

Mela[] è un sottotipo di Frutto[]

- A tempo di esecuzione otteniamo però un'**eccezione**:

```
Exception in thread "main" java.lang.ArrayStoreException: Pera
    at ViolenzaSuUnArray.prendiFrutta(ViolenzaSuUnArray.java:17)
    at ViolenzaSuUnArray.main(ViolenzaSuUnArray.java:9)
```

Wildcard come tipi generici

- Nel caso in cui non sia necessario utilizzare un tipo generico (ad es. T) nel corpo della classe o del metodo, è possibile utilizzare il jolly (wildcard) ?

```
public static void mangia(ArrayList<? extends Mangiabile> frutta)
{
    // mangia la frutta
}
```

- Equivalente a:

```
public static <T extends Mangiabile> void mangia2(ArrayList<T> frutta)
{
    // mangia la frutta
}
```

- ma **senza** la nozione del tipo utilizzato, ossia senza avere un simbolo T da usare nel corpo del metodo

Usare <?> per prendere in input un oggetto di una classe con qualsiasi tipo parametrico

- Nel caso in cui non sia necessario conoscere il tipo parametrico, si può utilizzare <?>
- Ad esempio:

```
public class Punto<T extends Number>
{
    private T x;
    private T y;

    public Punto(T x, T y) { this.x = x; this.y = y; }

    @Override
    public String toString() { return "("+x+";"+y+")"; }

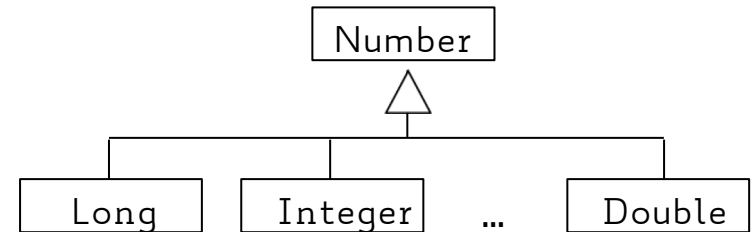
    public static void main(String[] args)
    {
        Punto<?> p = new Punto<Integer>(10, 42);
        System.out.println(p);

        p = new Punto<Double>(11.0, 43.5);
        System.out.println(p);
    }
}
```



Esempio: metodo generico di somma (1)

- Implementare un metodo generico **somma** che calcoli la somma di tutti i numeri contenuti in una collezione
- Implementazione **non generica**:



```
public class SommaNumeri
{
    public static void main(String[] args)
    {
        // interi e double
        Number[] numeri = { 1, 1.5, 2, 2.5 };
        ArrayList<Number> listaDiNumeri = new ArrayList<Number>();

        for (Number n : numeri)
            listaDiNumeri.add(n);

        System.out.printf("La lista contiene: %s\n", listaDiNumeri);
        System.out.printf("La somma dei numeri nella lista è: %.1f\n", somma(listaDiNumeri));
    }

    public static double somma(ArrayList<Number> lista)
    {
        double totale = 0.0;

        for (Number n : lista)
            totale += n.doubleValue();

        return totale;
    }
}
```

Posso creare **ArrayList<Number>**
contenenti interi, double, ecc. **misti**



Esempio: metodo generico di somma (2)

- Tuttavia, non posso passare in input un `ArrayList<Integer>`:

```
public static void main(String[] args)
{
    // interi
    Integer[] numeri = { 1, 2, 3, 4 };
    ArrayList<Integer> listaDiNumeri = new ArrayList<Integer>();

    for (Integer n : numeri)
        listaDiNumeri.add(n);

    System.out.printf("La lista contiene: %s\n", listaDiNumeri);
    System.out.printf("La somma dei numeri nella lista è: %.1f\n", somma(listaDiNumeri));
}
```

```
public static double somma(ArrayList<Number> lista)
{
    double totale = 0.0;

    for (Number n : lista)
        totale += n.doubleValue();

    return totale;
}
```



Esempio: metodo generico di somma (3)

Implementazione generica:

```
public class SommaNumeriGenerico
{
    public static void main(String[] args)
    {
        // interi
        Integer[] numeri = { 1, 2, 3, 4 };
        ArrayList<Integer> listaDiNumeri = new ArrayList<Integer>();

        for (Integer n : numeri)
            listaDiNumeri.add(n);

        System.out.printf("La lista contiene: %s\n", listaDiNumeri);
        System.out.printf("La somma dei numeri nella lista è: %.1f\n", somma(listaDiNumeri));
    }

    public static double somma(ArrayList<? extends Number> lista)
    {
        double totale = 0.0;

        for (Number n : lista)
            totale += n.doubleValue();

        return totale;
    }
}
```

Come funziona dietro le quinte?

- Mediante la **cancellazione del tipo** (type erasure)



- Quando il **compilatore** traduce il metodo/la classe generica in bytecode Java:
 1. **Elimina la sezione del tipo parametrico** e sostituisce il il simbolo nel codice con il tipo **Object** (a meno di vincoli sul tipo).
- **Solo una copia** del metodo o della classe viene creata!

Cancellazione del tipo: la classe Coppia

```
public class Coppia<T, S>
{
    private T a;
    private S b;

    public Coppia(T a, S b)
    {
        this.a = a;
        this.b = b;
    }

    public T getPrimo() { return a; }
    public S getSecondo() { return b; }
}
```



```
public class Coppia
{
    private Object a;
    private Object b;

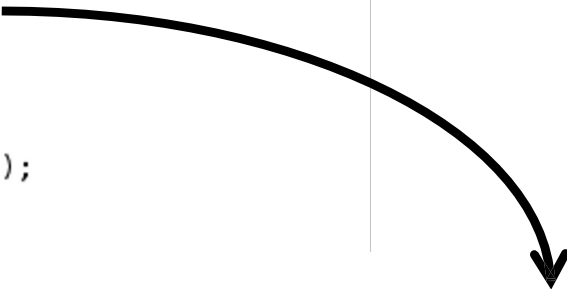
    public Coppia(Object a, Object b)
    {
        this.a = a;
        this.b = b;
    }

    public Object getPrimo() { return a; }
    public Object getSecondo() { return b; }
}
```

Cancellazione del tipo: il metodo massimo

```
public class Massimo
{
    static public <T extends Comparable<T>> T getMassimo(T a, T b, T c)
    {
        if (a.compareTo(b) > 0) return a.compareTo(c) >= 0 ? a : c;
        else return b.compareTo(c) >= 0 ? b : c;
    }

    public static void main(String[] args)
    {
        int max = getMassimo(10, 20, 30);
        String s = getMassimo("abc", "def", "ghi");
    }
}
```



```
public class Massimo
{
    static public Comparable getMassimo(Comparable a, Comparable b, Comparable c)
    {
        if (a.compareTo(b) > 0) return a.compareTo(c) >= 0 ? a : c;
        else return b.compareTo(c) >= 0 ? b : c;
    }

    public static void main(String[] args)
    {
        int max = (Integer)getMassimo(10, 20, 30);
        String s = (String)getMassimo("abc", "def", "ghi");
    }
}
```



Esercizio: cancellazione del tipo nella classe Pila

- Mostrare come viene cancellato il tipo dal compilatore nella classe Pila creata in un precedente esercizio
- Mostrare inoltre come viene trasformato il seguente codice:

```
Pila<Integer> p = new Pila<Integer>(10);  
p.push(1);  
p.push(2);  
Integer k = p.pop();
```

Come ottenere informazioni sull'istanza di un generico?

- Per via della cancellazione del tipo, non possiamo conoscere il tipo generico a tempo di esecuzione

```
public static void main(String[] args)
{
    List<Integer> x = Arrays.asList(4, 2);
    boolean b = x instanceof List<Integer>;
}
```

Cannot perform instanceof check against parameterized type List<Integer>. Use the form List<?> instead since further generic type information will be erased at runtime

Press 'F2' for focus

- Ma possiamo comunque verificarne il tipo usando il jolly?

```
public static void main(String[] args)
{
    List<Integer> x = Arrays.asList(4, 2);
    boolean b = x instanceof List<?>; // true
}
```

Le parole chiave `extends` e `super` nei generici

```
static public void esamina(ArrayList<Frutto> frutti)
{
    // ...
}
```

- Si può imporre un **vincolo** sul tipo generico `T` mediante la parola chiave:
 - **`extends`**: `T` deve essere un sottotipo della classe specificata o la classe stessa (**covarianza**)
 - **`super`**: `T` deve essere una superclasse della classe specificata o la classe stessa (**controvarianza**)

Interpretazione del vincolo extends

Permette di imporre il vincolo sul supertipo:

```
List<? extends Number> l1 = new ArrayList<Number>();
```

```
List<? extends Number> l2 = new ArrayList<Integer>();
```

```
List<? extends Number> l3 = new ArrayList<Double>();
```

- Non posso sapere a priori quali saranno i tipi "omogenei" di l1, l2 o l3, ma per certo so che saranno di tipo Number
- Non posso quindi assumere che l2 o l3 siano interi o double, perché il tipo parametrico di l2 o l3 non lo specifica

Interpretazione del vincolo super

Permette di imporre il vincolo sul sottotipo:

```
List<? super Integer> l1 = new ArrayList<Number>();
```

```
List<? super Integer> l2 = new ArrayList<Integer>();
```

```
List<? super Integer> l3 = new ArrayList<Object>();
```

- Non posso sapere a priori quali saranno i tipi nella lista, per cui posso solo assumere che saranno certamente **Object**

PECS: "Producer Extends, Consumer Supers"

- **extends** e **super** esistono per due necessità primarie:
 - Leggere da/scrivere in una collezione generica
- Ci sono 3 modi di utilizzare le wildcard«?»:
 - `List<?> l1 = new ArrayList<Number>();`
 - `List<? extends T> l2 = new ArrayList<Number>();`
 - `List<? super T> l3 = new ArrayList<Number>();`
- **<?> (il caso di l1)**: non so nulla sul tipo, quindi posso solo leggere con limitazione, non scrivere;
- **extends (il caso di l2)**: Se hai bisogno di una collezione in input che "produca" valori di tipo T, ma non puoi aggiungere elementi a questa lista;
- **super (il caso di l3)**: Se hai bisogno di una lista che consumi elementi di tipo T, per scriverli in una collezione, ma non puoi assumere il tipo degli stessi.

PECS: "Producer Extends, Consumer Supers"

- Letture di tipo **Object**: Quando devi solo leggere elementi e ti basta trattarli come istanze di **Object**.
- Metodi di utilità generica: Quando il codice utilizza solo metodi della classe **List** che non dipendono dal tipo parametrico (es. `size()`, `clear()`, `isEmpty()`).

```
// Questo metodo funziona con QUALSIASI tipo di lista (String, Integer, Personaggio,  
ecc.)  
public void printList(List<?> list) {  
    for (Object obj : list) {  
        System.out.println(obj); // Legge come Object  
    }  
    System.out.println("Totale elementi: " + list.size()); // Usa metodi  
indipendenti dal tipo  
}
```

PECS: "Producer Extends, Consumer Supers"

- Producer Extends (<? extends T>): Se hai bisogno di un List per produrre valori di tipo T (cioè li leggi), utilizza <? extends T>. Questo permette di passare una List<T> o una List<sottoclasse di T>

```
// Accetta una lista di Number o qualsiasi sottoclasse (Integer, Double)
public double sum(List<? extends Number> list) {
    double total = 0.0;
    for (Number n : list)
        total += n.doubleValue(); // Leggo da list
    return total;
}
```

PECS: "Producer Extends, Consumer Supers"

- Se hai bisogno di un List per consumare valori di tipo T (cioè li scrivi/aggiungi), utilizza <? super T>. Questo permette di passare una List<T> o una List<superclasse di T>

```
// Accetta una lista di Integer o qualsiasi superclasse (Number, Object)
public void addNumbers(List<? super Integer> list) {
    list.add(1); // Scrivo su list
    list.add(2);
}
```

Esempio: copia da una lista a un'altra seguendo il principio PECS

Il modo più **generale** e **sicuro** possibile di copiare con i generici da una lista di elementi di tipo `T` a un'altra lista è il seguente:

```
public <T> void copy(List<? extends T> src, List<? super T> dest)
{
    for (T k : src)
        dest.add(k);
}
```

dest è il **Consumer** (collezione sulla quale scriviamo solamente elementi)

`List<? super T>`

dest contiene elementi di tipo `T` o supertipo di `T`

`T`

src è il **Producer**
(collezione dalla quale leggiamo solamente elementi)

`List<? extends T>`

src contiene elementi di tipo `T` o sottotipo di `T`

Grazie all'applicazione di PECS non metterò mai una Pera in una collezione di Mele!

Overloading dei metodi generici

- Un metodo generico può essere sovraccaricato, come ogni altro metodo
- Anche da un metodo **non generico** con lo stesso nome e numero di parametri
- Quando il compilatore traduce una chiamata di metodo cerca il metodo più specifico
 - Prima il **non generico** e poi, eventualmente, il metodo generico

Esempio di overloading dei metodi generici

```
public class Massimo
{
    static public <T extends Comparable<T>> T getMassimo(T a, T b, T c)
    {
        if (a.compareTo(b) > 0) return a.compareTo(c) >= 0 ? a : c;
        else return b.compareTo(c) >= 0 ? b : c;
    }

    static public String getMassimo(String a, String b, String c)
    {
        // implementazione specifica per la stringa
        // ...
    }

    public static void main(String[] args)
    {
        // chiama l'implementazione generica
        int max = getMassimo(10, 20, 30);
        // chiama l'implementazione specifica per le stringhe
        String s = getMassimo("abc", "def", "ghi");
    }
}
```

I tipi raw

- I tipi **generici** sono stati introdotti con **Java 5**
- Per retrocompatibilità, è possibile istanziare una classe generica **senza specificare il tipo parametrico**

```
// istanza con tipo PARAMETRIZZATO  
Pila<Integer> p1 = new Pila<Integer>(10);
```

```
// istanza con tipo RAW  
Pila p2 = new Pila(10);
```

- E' possibile assegnare un'istanza con tipo parametrico a una con tipo raw:

```
// è possibile assegnare un'istanza  
// con tipo parametrico a una con tipo raw  
p2 = p1;
```

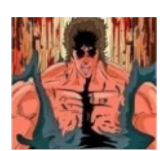
- E viceversa:

```
// e viceversa  
p1 = p2;
```



Esercizio: Successioni numeriche

- Scrivere una classe **SuccessioniNumeriche**, i cui oggetti sono inizialmente vuoti
- La classe espone un metodo **addSuccessione** che, preso in input un nome e una sequenza numerica, consente di aggiungere una successione (di lunghezza finita) del **tipo numerico** generico specificato (Integer, Float, Long, ecc.)
- La classe ha inoltre un metodo **getSuccessione**(String nome) con cui è possibile recuperare una successione a partire dal nome mnemonico
- Prevedere le seguenti successioni:
 - «Fibonacci»: 1, 1, 2, 3, 5, 8, 13, 21, 34
 - «1/n»: 1, 1/2, 1/3, 1/4, 1/5, 1/6
 - «RandomLong»: contiene oggetti Long costruiti casualmente
- **Hint:** utilizzare la classe **Number** come **superclasse** del tipo generico



Esercizio: multimappa generica

- Una multimappa è una mappa che ammette più valori a fronte di una data chiave
- Creare una classe **MultiMappa** generica sul tipo di chiavi e valori
- La classe implementa i seguenti metodi:
 - `put(k, v)` che associa il valore alla chiave specificata
 - `putAll(MultiMappa)` che aggiunge tutti gli elementi della multimappa in input alla mappa corrente
 - `removeAll(MultiMappa)` che rimuove tutte le chiavi della multimappa in input dalla mappa corrente
 - `get(k)` che restituisce l'insieme dei valori associati alla chiave
 - `get(k, p)`, come sopra ma restituisce solo i valori che soddisfano il predicato `p`
 - `values()` che restituisce l'elenco (con duplicati) dei valori contenuti nella multimappa
 - `valueSet()` che restituisce l'insieme dei valori contenuti nella multimappa
 - `transformToMultiMappa` che restituisce una multimappa in cui le coppie (k, v) sono trasformate in (k, z) secondo una funzione $(k, v) \rightarrow z$ (z può essere di tipo diverso rispetto a quello di v)
 - `mapEach` che sostituisce ciascun valore v con un valore dello stesso tipo secondo una funzione $(k, v) \rightarrow v'$
 - la classe è iterabile sulle coppie (k, v) mediante una classe interna **Elemento**

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.